SVN-to-Git Migration and Engineering Enablement

A production-oriented case study in source-control modernization, adoption, and team enablement

Author	Venkata Lakshmi Parimala Pasupuleti
Domain	Enterprise Software Engineering
System Area	Source Control and Developer Workflow
Status	Completed migration and team adoption
Primary Focus	Migration safety, knowledge transfer, and sustainable usage
Document Type	Engineering Case Study

Accuracy boundary

The verified experience is that the team used SVN, engineers were hesitant about Git, Venkata independently learned Git, conducted knowledge-transfer sessions, helped the team adopt the new workflow, and supported the product migration. Exact repository counts, command sequences, branching conventions, vendor tooling, dates, and quantitative productivity metrics were not supplied; they are therefore not presented as historical facts. Sections labeled "recommended" describe a defensible modern implementation, not claims about the original rollout.

Contents

1. Executive Summary

2. Context and Problem Statement

3. Scope, Goals, and Non-Goals

4. Legacy SVN Operating Model

5. Migration Principles

6. Readiness and Discovery

7. Repository Conversion Design

8. History, Identity, Branch, and Tag Integrity

9. Cutover and Rollback Strategy

10. Validation and Acceptance

11. Team Enablement and Knowledge Transfer

12. Git Working Model and Governance

13. Code Review and Collaboration

14. Operational Risks and Failure Modes

15. Security, Access, and Auditability

16. Adoption Measurement and Support

17. Outcomes and Engineering Impact

18. Lessons Learned

19. Recommended Modernization Direction

20. Operational Checklists

Appendix A. Terminology

Appendix B. Decision Record

1. Executive Summary

This case study describes the migration of an engineering team from Subversion (SVN) to Git and, more importantly, the organizational work required to make the new source-control model usable and sustainable. The legacy tool was familiar, centralized, and embedded in the team's daily habits. Git introduced a distributed model, local commits, explicit branching, merges, rebases, and a different mental model for history. The technical conversion mattered, but adoption risk was the dominant challenge.

Venkata's contribution began with self-directed learning. Although the migration was not initially assigned as his technology area, he studied Git independently, applied it in practice, translated the concepts into the team's working context, and conducted knowledge-transfer sessions. This reduced uncertainty and allowed engineers to develop confidence before and during the transition. The result was not merely a new repository; it was a change in how the team collaborated, reviewed code, isolated work, and reasoned about change history.

The migration illustrates a senior-engineering principle: infrastructure changes succeed only when the people operating them understand the new failure modes and have safe ways to practice. Converting commits is a finite technical task. Establishing repeatable workflows, recovery procedures, shared terminology, and support mechanisms is what turns a conversion into durable adoption.

The core engineering themes are controlled cutover, history integrity, identity mapping, branch and tag reconciliation, access governance, build verification, developer enablement, and post-cutover support. The document also distinguishes between the known historical experience and a recommended production-grade migration pattern that could be applied to a similar modernization today.

2. Context and Problem Statement

2.1 Organizational context

The engineering team had an established SVN workflow. Centralized version control was familiar, and team members had built muscle memory around update, commit, directory-based branching conventions, and a server-centered view of history. The migration to Git created uncertainty because many familiar commands had similar names but different semantics. Developers were concerned about losing work, creating merge conflicts, rewriting shared history, and making mistakes that would affect the team.

This hesitation was rational. A source-control migration changes a foundational system used by every engineer, every day. Small misunderstandings can block releases, create duplicate work, or damage confidence in the repository. The migration therefore needed both a technical plan and an enablement plan.

2.2 Problem statement

The team needed to transition from a centralized SVN repository to Git without losing meaningful history, disrupting active product work, or leaving developers unable to complete routine tasks. The desired outcome was a reliable Git repository and a team capable of using it safely for everyday development, collaboration, release support, and incident recovery.

2.3 Why the migration was difficult

- The source-control model changed from centralized to distributed.
- Every developer needed to learn a new mental model, not only new commands.
- Historical authors, branches, tags, and revisions needed to remain understandable.
- Active development could not stop indefinitely during conversion.
- Build and deployment processes depended on repository behavior.
- Errors during cutover could split the team across two sources of truth.
- The team required immediate support for merge conflicts, recovery, and workflow questions.



Figure 1. Migration sequence designed to reduce both repository risk and adoption risk.

3. Scope, Goals, and Non-Goals

3.1 Verified scope

- Independent learning and practical study of Git before team adoption.
- Knowledge-transfer sessions that explained Git concepts and workflows.
- Helping engineers move from uncertainty to practical daily usage.
- Supporting the product's source-control migration and team transition.
- Using influence and enablement rather than formal authority to drive adoption.

3.2 Engineering goals

Goal	Engineering meaning
Preserve trust	The migrated repository must retain understandable source history and ownership.
Avoid split-brain development	After cutover, engineers must know which repository is authoritative.
Minimize delivery disruption	Active product work and releases should resume quickly.

Goal	Engineering meaning
Create safe workflows	Developers need repeatable procedures for branches, commits, merges, and recovery.
Build team capability	The migration must reduce long-term dependence on one Git expert.
Support auditability	Changes should remain attributable and traceable.

3.3 Non-goals

The migration was not an attempt to redesign the product architecture, rewrite application code, or introduce every possible DevOps improvement at once. It was also not successful merely because a Git remote existed. The success condition was operational adoption: engineers could work safely, the product could continue shipping, and repository behavior remained understandable.

4. Legacy SVN Operating Model

SVN uses a centralized repository as the primary source of truth. Developers typically update a working copy, modify files, and commit directly to the central server. Branches and tags are represented as paths and are often created through directory copies. Revision numbers are global and linear across the repository. This model can be straightforward for teams accustomed to it, but it differs significantly from Git's object model and distributed history.

SVN concept	Git counterpart	Migration concern
Central revision number	Commit hash	No direct one-to-one global numbering model.
Repository path branch	Reference to a commit	Branch discovery and naming require interpretation.
Tag directory	Tag reference	Mutable SVN tag paths may not map cleanly to immutable release intent.
Central commit	Local commit plus push	Developers must understand unpublished local history.
Update working copy	Fetch/pull plus merge or rebase	The same goal has different history consequences.
Single server history	Distributed clones	Authority and remote conventions must be explicit.

The adoption challenge was therefore conceptual. Teaching a command-to-command translation would have been insufficient. Engineers needed to understand commits as immutable snapshots, branches as movable references, remotes as synchronization points, and the distinction between local history and shared history.

5. Migration Principles

5.1 One authoritative repository

The most important cutover principle is to avoid parallel authoritative histories. A controlled migration defines a final SVN synchronization point, publishes a validated Git repository, communicates the switch, and restricts or freezes writes to the legacy source after acceptance. Without that boundary, teams can create incompatible changes in both systems.

5.2 Validate meaning, not only file contents

A repository is more than its latest files. A trustworthy migration preserves enough history for engineers to answer who changed a behavior, when it changed, which release included it, and where a branch originated. Validation must therefore cover commit topology, authors, timestamps, tags, branch heads, and build behavior in addition to comparing the latest source tree.

5.3 Teach recovery before optimization

Early Git enablement should prioritize safe recovery: viewing status, inspecting diffs, creating a branch, restoring a file, aborting a merge, locating commits, and asking for help before a destructive operation. Advanced rebasing or history cleanup can wait. Confidence grows when developers know that most mistakes are recoverable.

5.4 Minimize simultaneous change

A source-control migration already changes developer behavior. Combining it with a new build system, branching model, release process, and repository decomposition can multiply risk. A safer approach establishes Git first, then introduces additional workflow improvements deliberately.

6. Readiness and Discovery

A production-grade migration begins with discovery. The purpose is to understand what the repository represents and what downstream systems assume about it. The exact discovery artifacts from the historical migration were not provided, so the following describes the recommended engineering approach consistent with the verified work.

6.1 Repository inventory

- Identify trunk, branch, and tag paths and determine whether conventions were consistently followed.
- Locate abandoned branches, release branches, vendor code, generated assets, binaries, and large files.
- Identify externals or cross-repository dependencies.
- Capture repository size, revision count, active authors, and recent activity.
- Document hooks, permissions, build jobs, deployment integrations, and scripts that reference SVN paths or revision numbers.

6.2 Author identity mapping

SVN usernames often need to be mapped to Git author names and email addresses. The mapping must be reviewed because authorship is part of auditability and debugging. Unknown or former users should receive stable placeholder identities rather than inconsistent values across commits.

6.3 Workflow observation

Migration planning should observe how engineers actually work: how releases are cut, how urgent fixes are applied, how branches are named, who can approve changes, and how build systems select a version. This prevents the migration design from optimizing for an imagined workflow rather than the operational one.

6.4 Training readiness

The team's confidence level is a technical risk indicator. Engineers should be able to complete a small set of tasks in a safe environment before cutover. Knowledge transfer is most effective when participants perform the workflow rather than only watch demonstrations.

7. Repository Conversion Design

A conversion tool reconstructs Git commits from SVN revisions and maps SVN paths to Git branches and tags. The conversion should be repeatable so that dry runs can be compared and the final import can be reproduced. Temporary manual edits to the converted repository should be avoided unless they are scripted and recorded.

7.1 Repeatable conversion pipeline

1. Create an immutable backup or verified mirror of the SVN repository.
2. Prepare the author identity mapping and path layout rules.
3. Run a full dry-run conversion into an isolated Git repository.
4. Generate validation reports for revisions, branch heads, tags, authors, and latest tree contents.
5. Review exceptions and update the conversion configuration.
6. Repeat until the output is stable and explainable.
7. Run the final synchronized conversion during the controlled cutover window.
8. Publish the accepted repository to the approved Git remote.

7.2 Repository structure decisions

Legacy SVN repositories may contain several products under one hierarchy. The migration must decide whether to preserve a monolithic repository or split components. Splitting can improve ownership boundaries but may complicate history, shared code, atomic changes, and release coordination. Because the historical repository topology was not supplied, this document does not claim which option was used.

7.3 Binary and generated content

Large binaries and generated artifacts can make Git clones expensive. The migration should identify whether these files belong in source control, in Git LFS, or in an artifact repository. The

decision should consider reproducibility, build dependencies, and the ability to retrieve historical releases.

7.4 Commit-message traceability

When useful, migrated commit messages can retain the original SVN revision identifier. This creates a bridge for old ticket references and operational notes that mention an SVN revision. The mapping should be deterministic and documented rather than added inconsistently.

8. History, Identity, Branch, and Tag Integrity

8.1 History integrity

History validation should compare the converted repository with the SVN source at selected revisions and important releases. The objective is not to force Git to look exactly like SVN; it is to preserve the information engineers need to reason about product evolution.

Validation dimension	Example check	Failure signal
Latest content	Compare file lists and checksums at authoritative heads.	Missing, extra, or changed files.
Commit coverage	Compare imported revision ranges and exception lists.	Unexpected gaps or duplicated imports.
Authors	Review mapped identities and unknown users.	Blank, inconsistent, or incorrect attribution.
Branches	Compare active branch heads and ancestry.	Missing branch or incorrect source point.
Tags/releases	Reconcile release labels with known release artifacts.	Tag points to wrong code state.
Timestamps	Sample historical commits across periods.	Time-zone shifts or ordering anomalies.
Buildability	Build selected current and historical states.	Migration altered required source or metadata.

8.2 Branch interpretation

SVN branches are copied directory trees, while Git branches are references to commits. Conversion tools infer relationships from copy history, but unconventional SVN usage can produce unexpected Git topology. Active branches and release branches deserve explicit review. Obsolete branches can remain available as archival references without becoming part of the everyday workflow.

8.3 Tag integrity

SVN does not technically prevent changes under a tag path. A migration should detect tags that were modified after creation. Teams can preserve the final tag state, annotate exceptions, and establish immutability expectations in Git. Release traceability is more important than pretending legacy data was cleaner than it was.

9. Cutover and Rollback Strategy

The cutover is the point at which the Git repository becomes authoritative. It should be treated as a production change because it affects every engineer and potentially the build and deployment path. The plan must define owners, a communication channel, an acceptance checklist, and a rollback window.

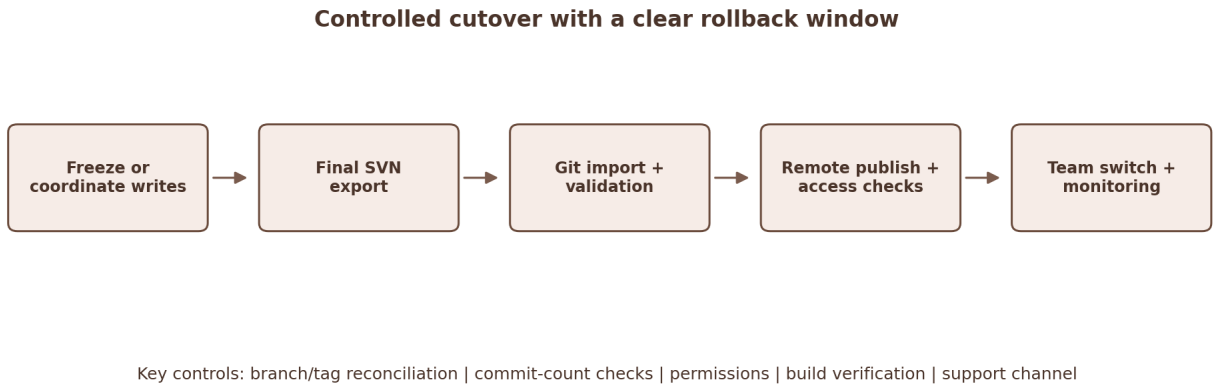


Figure 2. Recommended controlled cutover sequence.

9.1 Pre-cutover controls

- Complete at least one end-to-end rehearsal.
- Publish the migration runbook and developer quick-start guide.
- Confirm author mapping and repository access lists.
- Coordinate active branches and urgent production work.
- Back up the SVN repository and conversion configuration.
- Preconfigure the Git remote, protections, and integrations where possible.
- Establish a temporary support channel for cutover questions.

9.2 Cutover execution

A safe cutover coordinates final writes, performs the last import, validates repository state, publishes the remote, and runs build verification. Engineers should not begin new work in Git until acceptance is declared. The legacy repository should then be made read-only or clearly marked as archived to prevent accidental divergence.

9.3 Rollback

Rollback means returning the team to the known SVN source of truth before incompatible Git-only work accumulates. The rollback trigger should be explicit: missing history that affects releases, repository corruption, failed build/deployment integration, or access failures that block a meaningful portion of the team. Minor usage questions are support issues, not automatic reasons to abandon the cutover.

10. Validation and Acceptance

Acceptance should combine automated checks and human review. Automated checks find count and content mismatches; human reviewers recognize whether branches, tags, and product history make sense. The final repository should be tested through realistic developer and delivery workflows.

10.1 Technical acceptance criteria

- The default branch contains the expected authoritative product state.
- Known releases map to the correct Git tags or documented equivalents.
- Active branches are present and their heads reconcile with SVN.
- Author identities are mapped consistently.
- Builds and tests run successfully from a clean Git clone.
- Repository permissions and authentication work for intended users.
- CI or deployment integrations can resolve the required branches and commits.
- The legacy repository is protected from unintended new authoritative changes.

10.2 Developer acceptance scenarios

9. Clone the repository and configure identity.
10. Create a branch from the agreed base.
11. Make and review a local commit.
12. Synchronize with the remote and resolve a controlled conflict.
13. Push the branch and open the team's review mechanism.
14. Apply requested changes and update the branch.
15. Locate historical changes by file, author, and commit message.
16. Recover from a deliberately introduced local mistake.

10.3 Evidence retention

Migration reports, validation outputs, author maps, conversion scripts, and acceptance decisions should be retained. These artifacts explain how the repository was produced and reduce uncertainty if a historical discrepancy is found later.

11. Team Enablement and Knowledge Transfer

The defining element of Venkata's contribution was translating independent learning into team capability. The team's concern was not solved by announcing new commands. Engineers needed a mental model, opportunities to practice, and a safe path for asking questions. Knowledge-transfer sessions made the transition operational rather than theoretical.

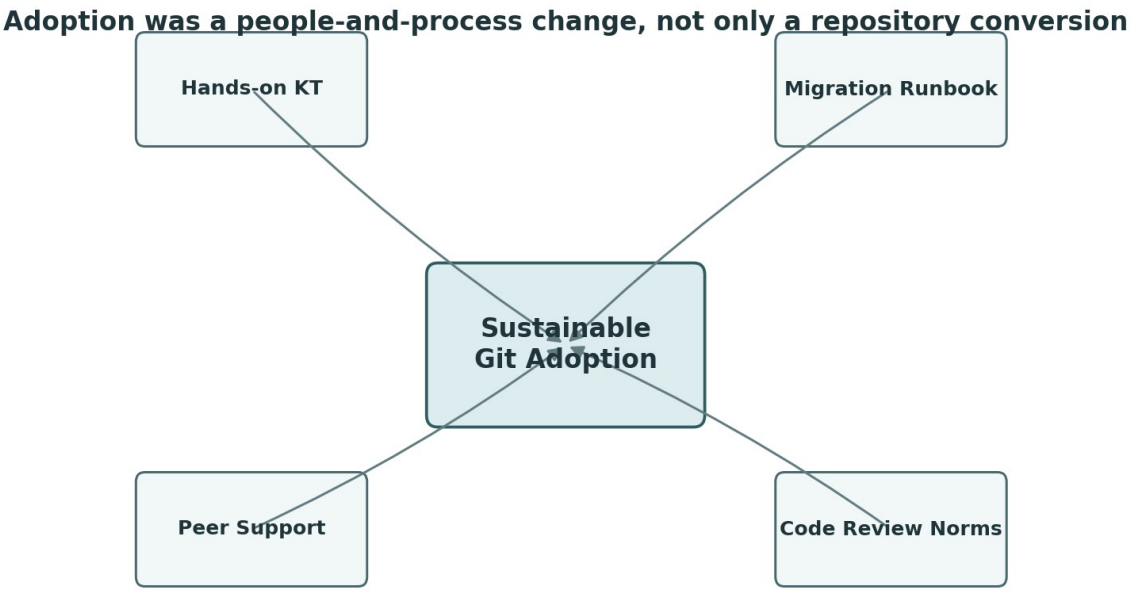


Figure 3. Sustainable adoption depends on training, documentation, peer support, and shared norms.

11.1 Training sequence

A practical sequence begins with the repository model and moves toward everyday work. The goal is to explain why Git behaves as it does so that developers can diagnose unfamiliar situations without memorizing isolated commands.

Training module	Core capability	Reason
Mental model	Working tree, index, local commits, branches, remote references.	Prevents incorrect command analogies with SVN.
Daily workflow	Status, diff, add, commit, fetch, pull, push.	Covers the majority of routine work.
Branching	Create, switch, update, merge, delete.	Enables isolated and reviewable changes.
Conflict resolution	Read conflict markers, choose content, test, continue or abort.	Reduces fear and unsafe improvisation.
Recovery	Restore files, inspect reflog, reset safely, revert shared changes.	Builds confidence that mistakes are recoverable.
Collaboration	Review, shared-history rules, commit hygiene.	Establishes consistent team behavior.

11.2 Hands-on practice

Demonstrations should be paired with a sandbox exercise. Each engineer should create a branch, commit a change, synchronize with a remote, resolve a controlled conflict, and recover from a mistake. This exposes misunderstandings before they affect product work.

11.3 Office hours and peer support

During early adoption, fast support prevents developers from reverting to unsafe workarounds. A temporary help channel, office hours, and peer review of unfamiliar commands can reduce risk. The long-term goal is distributed capability, not permanent dependence on the migration champion.

11.4 Documentation

A concise runbook is more useful than a generic Git manual. It should document the team's actual branch names, review path, release behavior, conflict-escalation path, and recovery expectations. Command examples should be accompanied by the intended result and warnings about destructive options.

12. Git Working Model and Governance

Status of this section

The exact branch model used in the original migration was not provided. The following is a recommended working model for a product team moving from SVN. It is included as technical design guidance, not as a historical claim.

12.1 Default branch

The default branch should have a clear meaning: deployable mainline, integration branch, or another explicitly defined state. Ambiguity about the default branch produces inconsistent feature bases and release behavior. Protection should prevent accidental direct updates if the team relies on reviewed changes.

12.2 Short-lived branches

Short-lived feature or fix branches reduce long-running divergence. Engineers should synchronize frequently and keep changes focused. The branch name can include a ticket identifier when that improves traceability, but naming conventions should remain simple enough to follow consistently.

12.3 Merge versus rebase

The team should define when rebasing is safe. Rewriting private, unpublished commits can simplify a branch. Rewriting shared history can disrupt collaborators and should be restricted. The important rule is not ideological preference; it is predictable behavior and clear ownership of shared references.

12.4 Release and hotfix handling

Release branches and hotfix paths should reflect the product's actual delivery model. A regulated or enterprise product may need supported release lines, selective fixes, and traceable tags. The process should identify where a fix originates, which versions receive it, and how the change returns to the main line to prevent regression.

12.5 Protected references

- Require review or approved automation before updating protected branches.
- Prevent force pushes to shared release and default branches.
- Restrict tag creation or deletion when tags identify production releases.
- Use least-privilege access for repository administration.
- Retain audit logs for sensitive changes to permissions and protected references.

13. Code Review and Collaboration

Git creates opportunities for branch-based review, but a review tool alone does not improve quality. The team must establish expectations for change size, context, testing evidence, reviewer responsibility, and response time. The migration can be used to strengthen collaborative engineering without turning review into a bureaucratic gate.

13.1 Change design

Reviewable changes are cohesive and explain why the change exists. Large mixed-purpose commits make debugging and rollback difficult. Engineers should separate formatting, refactoring, schema changes, and behavior changes when practical, while avoiding artificial fragmentation that hides the complete impact.

13.2 Commit quality

A useful commit records a meaningful product or engineering step and leaves the repository in an understandable state. Commit messages should explain intent and trade-offs rather than restating filenames. Perfectly curated history is less important than reliable traceability and safe collaboration.

13.3 Review responsibility

Reviewers validate correctness, maintainability, operational impact, and consistency with system behavior. Authors remain responsible for testing and explaining the change. Review should not become a transfer of ownership from author to reviewer.

14. Operational Risks and Failure Modes

Risk	Impact	Detection	Mitigation
Split-brain commits	Different changes land in SVN and Git.	Compare post-freeze activity and team reports.	Freeze or archive SVN; communicate authority clearly.

Risk	Impact	Detection	Mitigation
Missing branch or tag	Release history becomes incomplete.	Branch/tag reconciliation report.	Dry-run conversion; explicit exception review.
Incorrect author map	Audit and debugging attribution degrade.	Unknown-author report and samples.	Reviewed deterministic identity mapping.
Changed timestamps or topology	Historical investigation becomes confusing.	Sample commit validation across periods.	Document conversion semantics and exceptions.
Build integration failure	Delivery is blocked after cutover.	Clean-clone build and pipeline test.	Test integrations before declaring acceptance.
Access failure	Engineers cannot clone or push.	User acceptance and permission tests.	Pre-provision access and support escalation.
Unsafe history rewrite	Shared work appears lost or diverges.	Remote audit and user reports.	Protected branches; training; recovery runbook.
Large repository performance	Clone and CI latency increase.	Repository size and pipeline duration.	Remove generated assets; use LFS or artifacts where justified.
Low adoption confidence	Engineers create workarounds or avoid collaboration.	Support volume and repeated workflow errors.	Hands-on KT, office hours, peer support.

14.1 Conflict handling

Merge conflicts are not inherently failures; they indicate overlapping changes that require a decision. The risk comes from resolving them without understanding either side. The runbook should require inspection, targeted testing, and escalation for conflicts in schema, configuration, financial logic, or other high-impact areas.

14.2 Recovery philosophy

Recovery guidance should distinguish unpublished local history from shared remote history. Local mistakes can often be repaired through restore, reset, or reflog-based recovery. Shared mistakes should generally be corrected with auditable new commits, such as revert, unless repository administrators deliberately coordinate another action.

15. Security, Access, and Auditability

Source control contains intellectual property, credentials risk, customer-impacting logic, and release history. Migration should preserve or improve access governance. Repository administration must be limited, and the team should understand that distributed clones create additional endpoints containing source history.

- Map legacy access groups to the Git hosting platform using least privilege.
- Require approved authentication mechanisms and rotate migration credentials.
- Review repositories for secrets before and after conversion.
- Protect default and release references from destructive updates.
- Retain administrative audit events and access-change history.
- Define offboarding procedures that remove remote access while recognizing existing local clones.
- Avoid storing build artifacts, secrets, or customer data in the repository.

If secret scanning identifies a historical credential, deleting it from the latest branch is insufficient because Git preserves history. Remediation requires credential rotation first, followed by a deliberate history-cleanup decision and communication to all clone owners.

16. Adoption Measurement and Support

No quantitative adoption metrics were supplied for the original migration. A modern rollout should measure signals that reveal whether the new workflow is usable and stable. These metrics are diagnostic, not performance scores for individual engineers.

Signal	What it indicates	Example response
Clone/access failures	Provisioning or authentication friction.	Fix onboarding and access automation.
Repeated merge-conflict escalations	Training or branch-lifetime problem.	Targeted workshops; encourage smaller branches.
Direct protected-branch attempts	Workflow misunderstanding or missing automation.	Clarify review path and permissions.
Build failures after migration	Integration or environment assumptions.	Reproduce from clean clone and patch pipeline.
High support-question concentration	Knowledge remains centralized.	Develop additional peer champions.
Long-lived inactive branches	Cleanup and ownership ambiguity.	Establish archival policy.
Review cycle time	Collaboration bottleneck or oversized changes.	Adjust ownership and change sizing.

16.1 Support model

A short period of elevated support is normal. Questions should be captured and converted into runbook improvements. The support model should intentionally decay as capability spreads across the team. If every decision continues to require the migration champion, enablement is incomplete.

17. Outcomes and Engineering Impact

The migration established Git as the team's source-control platform and helped the product team move beyond hesitation into practical usage. Venkata's self-directed learning and knowledge-

transfer sessions were central to adoption. The impact was not limited to tooling; the team gained a stronger basis for isolated work, collaborative review, clearer change history, and future automation.

The case demonstrates technical influence without formal authority. Venkata recognized a capability gap, developed expertise independently, translated it into the team's context, and helped others operate the new system. That is a senior-engineering behavior because it improves the effectiveness of the entire team rather than only completing an assigned code change.

Most important outcome
The durable result was not that developers memorized Git commands. It was that the team developed enough shared understanding to use the system safely, recover from mistakes, and continue product delivery without depending on fear-driven workarounds.

18. Lessons Learned

18.1 Technology adoption is a systems problem

A migration changes tools, mental models, permissions, build integrations, release processes, and team habits at the same time. Treating it as a file conversion underestimates the operational risk.

18.2 Confidence comes from recovery skills

Developers become comfortable when they understand how to inspect state and recover. Teaching only the happy path leaves the team vulnerable to the first unexpected conflict or mistaken command.

18.3 Influence scales through enablement

One engineer learning Git solves a personal problem. Turning that knowledge into sessions, examples, runbooks, and peer capability solves a team problem. Knowledge transfer is therefore an engineering deliverable.

18.4 History must remain explainable

A perfect mechanical translation is less important than an explainable repository. Exceptions should be documented. Engineers must know what history means after conversion and where legacy semantics differ.

18.5 Cutover needs an authority boundary

The team must know exactly when Git becomes authoritative. A migration without a clear source-of-truth transition invites divergence and undermines trust.

18.6 Avoid unnecessary simultaneous transformation

Establish the new source-control foundation first. Branch policy, CI improvements, repository restructuring, and release automation can be introduced in deliberate increments after the team is stable.

19. Recommended Modernization Direction

Design status

This section is a forward-looking architecture and operating model. It should not be represented as functionality or infrastructure that was necessarily present in the historical migration.

19.1 Repository-as-product ownership

Assign explicit ownership for repository settings, permissions, branch protections, templates, and integration health. Source control is a production dependency and deserves operational stewardship rather than ad hoc administration.

19.2 Automated policy and validation

- Automate clean-clone builds and tests on reviewed changes.
- Use secret scanning and dependency analysis appropriate to the product.
- Validate protected-branch and release-tag policies through platform configuration.
- Generate release notes or change summaries from traceable metadata.
- Archive stale branches through a transparent ownership process.
- Monitor Git hosting availability and CI queue health as delivery dependencies.

19.3 Scalable onboarding

Convert the original KT material into an onboarding path with a sandbox repository, guided exercises, team-specific workflow documentation, and a short recovery lab. New engineers should learn the operating model before working on high-risk product code.

19.4 Migration reproducibility

Store conversion configuration, identity maps, validation scripts, and reports in a controlled engineering repository. A reproducible migration is easier to audit, troubleshoot, or repeat for related repositories.

19.5 Better traceability

Integrate ticket identifiers, reviewed changes, CI evidence, release tags, and deployment records without forcing unnecessary process. The objective is to make customer-impacting changes easy to trace from requirement through production.

20. Operational Checklists

20.1 Pre-migration checklist

- Inventory repository layout, branches, tags, externals, large files, and active users.
- Back up the SVN source and verify restore access.
- Create and review the author identity map.
- Document build, release, hook, and deployment dependencies.
- Complete dry-run conversion and exception review.
- Prepare training material, quick-start guidance, and support channel.
- Confirm Git remote, permissions, protections, and integrations.
- Define cutover owner, acceptance owner, and rollback criteria.

20.2 Cutover checklist

- Coordinate or freeze final SVN writes.
- Run final synchronized import using the approved configuration.
- Validate default branch, active branches, tags, authors, and content.
- Run clean-clone build and integration checks.
- Publish acceptance decision and authoritative-repository notice.
- Make SVN read-only or clearly archived.
- Open elevated support coverage and track issues.

20.3 Post-cutover checklist

- Review access, clone, push, and build failures.
- Capture recurring questions and update the runbook.
- Validate release and urgent-fix workflows.
- Confirm no authoritative work continued in SVN.
- Review stale or incorrectly migrated references.
- Distribute Git knowledge across additional team members.
- Close migration support only after routine workflows are stable.

Appendix A. Terminology

Term	Meaning in this document
Working tree	The checked-out files a developer edits.
Index/staging area	The proposed content of the next Git commit.
Commit	An immutable snapshot with parent relationship, author, timestamp, and message.
Branch	A movable reference to a commit.
Tag	A named reference, commonly used for a release or milestone.
Remote	A named location used to exchange Git references and objects.
Fetch	Update remote-tracking references without

Term	Meaning in this document
	modifying the current branch.
Merge	Combine histories and record the integration.
Rebase	Replay commits on another base, rewriting those commit identities.
Revert	Create a new commit that reverses an earlier change.
Force push	Update a remote reference in a way that can replace shared history.
Cutover	The controlled transition in which Git becomes authoritative.

Appendix B. Decision Record

Decision	Rationale	Trade-off
Treat enablement as part of migration	Every developer depends on source control; usage errors are operational risks.	Training and support add effort before and after cutover.
Use dry-run conversions	Allows history exceptions to be found before the final transition.	Requires repeatable tooling and additional elapsed time.
Define one authority boundary	Prevents incompatible commits in SVN and Git.	Requires coordination or a brief write freeze.
Prioritize recovery training	Reduces fear and unsafe experimentation.	Advanced optimization topics are deferred.
Validate history and builds	Source files alone do not prove migration correctness.	Broader acceptance requires cross-functional participation.
Separate modernization from historical claims	Maintains credibility and technical accuracy.	Some exact implementation detail remains intentionally unspecified.

End of Case Study